MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITE DE SOUSSE

INSTITUT SUPERIEUR D'INFORMATIQUE ET DES TECHNOLOGIES DE COMMUNICATION

Rapport de projet de fin d'annnée

Présenté en vue de l'obtention du diplôme de Licence en Science de l'Informatique (Computer Science)

Spécialité : Informatique et Multimédia

Conception et développement d'une Plateforme "WinLkar"

Réalisé par :

Alaa Eddine Kerkeni
Sadok Belhadj
Abdelaziz Youssef
Ahmed Neji
Mohamed Salah Mejri
Mohamed Karmous

Année universitaire : 2025/2026

Institut Supérieur d'Informatique et des Technologies de Communication
ISITCOM
Tél/Fax : +216 73 37 15 71 / +216 73 36 44 11

Remerciements

Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année. Ce travail n'aurait pas pu voir le jour sans le soutien, les conseils et l'accompagnement bienveillant de nombreuses personnes.

Nous adressons tout d'abord nos sincères remerciements à notre encadrant pédagogique (Amel Elbehi), qui a fait preuve d'une grande disponibilité et d'une patience remarquable tout au long de ce projet. Ses orientations précieuses, ses remarques constructives et son suivi rigoureux nous ont permis d'avancer dans les meilleures conditions possibles.

Nous remercions également l'ensemble du corps enseignant de l'Institut Supérieur d'Informatique et des Technologies de Communication (ISITCOM) de Sousse, dont la qualité de l'enseignement dispensé durant nos années de formation nous a fourni les bases théoriques et pratiques indispensables à la réalisation de ce projet.

Enfin, nous remercions tous nos camarades et amis qui ont partagé avec nous les moments difficiles comme les moments de réussite, et dont l'entraide et la solidarité ont rendu ce parcours encore plus enrichissant.

Table des matières

Remerciements **1**

Introduction générale **4**

1 Cadre général du projet **5**
1.1 Introduction 5
1.2 Contexte du projet 5
1.3 Problématique 6
1.4 Objectifs du projet 6
1.5 Étude de l'existant 6
1.5.1 Présentation des solutions existantes 6
1.5.2 Tableau comparatif 7
1.5.3 Conclusion de l'analyse 7
1.6 Solution Proposée 7
1.7 Environnement et Outils de Développement 8
1.7.1 Android Studio 8
1.7.2 Adobe Photoshop 8
1.7.3 GitHub et Git 8
1.7.4 Firebase 8
1.7.5 Google Maps Platform 8
1.7.6 Tableau récapitulatif des outils 9
1.8 Conclusion 9

2 Conception et spécification du projet **10**
2.1 Introduction 10
2.2 Organisation de l'Équipe 10
2.2.1 Composition et rôles 10
2.2.2 Outils de communication et de collaboration 10
2.3 Mise en place de l'environnement de développement 11
2.3.1 Installation et configuration des outils 11
2.4 Architecture Technique 13
2.4.1 Vue d'ensemble de l'architecture 13
2.4.2 Stack technologique 14
2.5 Diagramme de Cas d'Utilisation Global 14
2.6 Diagramme de Classes 15
2.7 Diagramme de Séquence — Authentification 16
2.8 Diagramme de Séquence — Mise à jour GPS 17
2.9 Modélisation des Données 18
2.9.1 Entités principales 18

2.9.2 Schéma Firebase Realtime Database . . . 18
2.10 Besoins Fonctionnels . . . 19
2.10.1 Besoins du Passager . . . 19
2.10.2 Besoins du Conducteur . . . 19
2.10.3 Besoins de l'Administrateur . . . 19
2.11 Besoins Non Fonctionnels . . . 20
2.12 Product Backlog . . . 20
2.13 Méthodologie de Développement — Scrum . . . 21
2.13.1 Les rôles Scrum dans WinLkar . . . 21
2.13.2 Planification des Sprints . . . 22
2.14 Conclusion . . . 22

3 Réalisation — Sprint 1 23
3.1 Introduction . . . 23
3.2 Backlog du Sprint 1 . . . 23
3.3 Conception . . . 24
3.3.1 Diagramme de Cas d'Utilisation — Sprint 1 . . . 24
3.3.2 Diagrammes de Séquence — Sprint 1 . . . 25
3.4 Réalisation . . . 28
3.5 Interface Passager . . . 28
3.5.1 Liste des Stations . . . 28
3.5.2 Carte interactive et suivi en temps réel . . . 28
3.6 Code Source Java — Extraits Essentiels . . . 28
3.6.1 PassengerActivity.java — Listener Firebase temps réel . . . 29
3.6.2 Calcul de l'ETA — Formule de Haversine . . . 29
3.7 Interface Conducteur . . . 29
3.7.1 Fonctionnalités implémentées . . . 29
3.8 Calcul de l'ETA . . . 29
3.9 Modèle de Données Firebase . . . 30
3.10 Tests et Validation . . . 30
3.11 Sprint 1 Review . . . 30
3.12 Conclusion . . . 30

4 Réalisation — Sprint 2 31
4.1 Introduction . . . 31
4.2 Backlog du Sprint 2 . . . 31
4.3 Conception . . . 32
4.3.1 Diagramme de Cas d'Utilisation — Sprint 2 . . . 32
4.3.2 Architecture Backend — Diagramme de Séquence JWT . . . 34
4.4 Réalisation . . . 34
4.4.1 Authentification et sécurité . . . 34
4.5 Interface Administrateur . . . 35
4.5.1 Fonctionnalités de l'AdminActivity . . . 35
4.5.2 Portail d'accès administrateur . . . 35
4.6 Structure de la Base de Données Firebase . . . 35
4.7 Service de Géolocalisation . . . 36
4.8 Service ETA Backend . . . 36
4.9 API REST — Endpoints principaux . . . 36
4.10 Déploiement avec Google Cloud Platforme . . . 37

4.11 Tests et Documentation 37
4.11.1 Documentation de Google Cloud 37
4.12 Sprint 2 Review 37
4.13 Conclusion 37

Conclusion Générale **38**

Bibliographie et Webographie **40**

A Annexe A — Configuration Firebase **41**
A.1 google-services.json 41
A.2 Règles de sécurité Firebase 41

B Annexe B — Description des Interfaces de l'Application **42**

C Annexe C — Glossaire **43**

Introduction générale

Dans le cadre de la transformation numérique des services de transport public, le suivi en temps réel des véhicules constitue une problématique cruciale pour l'amélioration de l'expérience utilisateur. En effet, les usagers des transports en commun font face quotidiennement à des difficultés liées à l'incertitude des horaires, à l'absence d'information sur la position des bus, et à la difficulté de planifier leurs déplacements de manière efficace.

Dans la région de Sousse notamment, les passagers n'ont aucun moyen numérique de savoir quand leur bus arrivera à l'arrêt. Cette lacune génère de la frustration, des temps d'attente prolongés et une sous-utilisation des transports en commun au profit de modes de transport individuels, ce qui aggrave la congestion et la pollution.

C'est dans ce contexte que s'inscrit notre projet de fin d'études intitulé **WinLkar**, une application mobile Android de suivi de bus en temps réel. L'objectif principal est de concevoir et développer une solution complète permettant aux passagers de visualiser la position des bus actifs sur une carte interactive, d'estimer le temps d'arrivée aux arrêts et d'obtenir des informations détaillées sur chaque trajet.

L'application repose sur une architecture moderne combinant une application native Android développée en Java, une base de données Firebase Realtime Database pour la synchronisation en temps réel des positions GPS, et une API REST Node.js avec Socket.IO pour la gestion avancée des utilisateurs, des trajets et des réclamations.

Notre approche méthodologique est basée sur **Scrum**, une méthode agile permettant un développement itératif et incrémental organisé en sprints, favorisant la collaboration et la livraison régulière de fonctionnalités.

Ce rapport est structuré en cinq chapitres principaux :

- **Chapitre 1** : présente le cadre général du projet, la problématique et les objectifs.
- **Chapitre 2** : décrit la conception et la spécification, incluant les diagrammes UML.
- **Chapitre 3** : présente la méthodologie Scrum et la planification des sprints.
- **Chapitre 4** : détaille la réalisation technique du Sprint 1 (application Android).
- **Chapitre 5** : présente le Sprint 2 (backend API et fonctionnalités avancées).

Chapitre 1

Cadre général du projet

1.1 Introduction

Ce premier chapitre pose le cadre général de notre projet de fin d'année. Nous y présentons le contexte dans lequel s'inscrit WinLkar, la problématique identifiée dans le domaine du transport urbain en Tunisie, les objectifs que nous nous sommes fixés, une étude comparative des solutions existantes, et enfin la solution que nous proposons ainsi que les outils de développement retenus.

1.2 Contexte du projet

Le transport en commun joue un rôle essentiel dans la mobilité urbaine, notamment dans des villes à forte densité comme Sousse, deuxième ville de Tunisie avec plus de 700 000 habitants dans son grand gouvernorat. Cependant, l'absence de systèmes d'information en temps réel sur la position des véhicules génère de l'incertitude et de la frustration chez les usagers.

La pénétration des smartphones en Tunisie dépasse 80% de la population selon les dernières statistiques, ce qui crée un terreau favorable au développement d'applications mobiles utiles. Pourtant, l'offre d'applications de transport en commun adaptées à la réalité tunisienne reste extrêmement limitée.

Les utilisateurs des bus à Sousse et dans les villes avoisinantes (Monastir, Hammam Sousse, Akouda, Port El Kantaoui) doivent se fier aux affichages aux arrêts, souvent obsolètes, ou au bouche-à-oreille pour connaître les horaires. Cette situation est d'autant plus problématique pour les touristes qui fréquentent la région en grand nombre.

Du côté des conducteurs, la gestion des trajets reste manuelle et peu coordonnée. Il n'existe aucun outil numérique permettant aux chauffeurs de communiquer efficacement leur position GPS aux passagers, ni de gérer les incidents en temps réel. Les centrales de bus ne disposent d'aucun tableau de bord permettant de visualiser l'ensemble de la flotte en temps réel.

1.3 Problématique

La problématique principale peut se formuler ainsi : comment concevoir une application mobile permettant de suivre en temps réel la position des bus, d'informer les passagers sur les trajets actifs et les temps d'arrivée estimés, tout en offrant aux conducteurs les outils nécessaires pour gérer efficacement leurs trajets, et aux administrateurs un tableau de bord centralisé ?

Les problèmes spécifiques identifiés sont les suivants :

— Absence d'information en temps réel sur la position des bus pour les passagers.

— Impossibilité pour les passagers d'estimer le temps d'arrivée du bus à leur arrêt.

— Manque d'outil numérique pour les conducteurs pour déclarer et gérer leurs trajets.

— Absence de système de gestion centralisé pour les administrateurs.

— Inexistence de mécanisme de réclamations et d'évaluation des trajets.

— Absence de données cartographiques spécifiques au réseau de bus de Sousse.

1.4 Objectifs du projet

L'objectif principal est de développer **WinLkar**, une application mobile Android complète offrant les fonctionnalités suivantes :

— Permettre aux passagers de voir en temps réel tous les bus actifs sur une carte interactive Google Maps.

— Calculer et afficher le temps d'arrivée estimé (ETA) pour chaque station d'un trajet.

— Permettre aux conducteurs de déclarer le début et la fin d'un trajet et de partager leur position GPS.

— Offrir un système d'authentification sécurisé avec gestion des rôles (passager, conducteur, administrateur).

— Implémenter un système de notation et de réclamations pour les trajets.

— Fournir un tableau de bord administrateur pour la gestion complète du système (bus, conducteurs, trajets, feedbacks).

1.5 Étude de l'existant

1.5.1 Présentation des solutions existantes

Google Maps Transit est l'application générale de navigation la plus utilisée au monde. Elle intègre les données de transport en commun dans certaines villes, mais ne propose pas de suivi en temps réel des bus dans les villes tunisiennes. Les données affichées sont statiques et basées sur des horaires préenregistrés.

Moovit est une application spécialisée dans la planification de trajets en transport en commun. Elle agrège les données de plusieurs opérateurs à travers le monde. Cependant, elle n'est pas disponible pour la Tunisie et ne propose pas d'interface dédiée aux conducteurs.

Trafi est une application similaire à Moovit, présente dans quelques villes européennes et d'Amérique latine. Elle n'est pas disponible en Tunisie et son modèle repose exclusivement sur des partenariats avec des opérateurs de transport.

Waze Carpool se concentre sur le covoiturage et ne couvre pas les transports en commun.

1.5.2 Tableau comparatif

TABLE 1.1 – Comparaison des solutions existantes avec WinLkar

Critère	Google Maps	Moovit	Trafi	**WinLkar**
Suivi temps réel des bus	**Non**	Partiel	Partiel	
Disponible en Tunisie	Partiel	**Non**	**Non**	
Interface conducteur	**Non**	**Non**	**Non**	
Gestion administrateur	**Non**	**Non**	**Non**	
ETA par arrêt	**Non**			
Système de réclamations	**Non**	**Non**	**Non**	
API REST ouverte	**Non**	**Non**	**Non**	
Gratuit pour l'utilisateur				

1.5.3 Conclusion de l'analyse

Cette analyse comparative montre clairement que les plateformes générales (STAGI, StageConnect, Stage Tunisie) se concentrent uniquement sur la mise en relation entre étudiants et entreprises, sans offrir de fonctionnalités de suivi académique interne. Les plateformes institutionnelles (FMT, ESPRIT) proposent une gestion plus complète mais restent fermées et spécifiques à leurs établissements respectifs. Notre solution se distingue en combinant les deux aspects : elle offre à la fois un suivi académique complet (demandes d'encadrement, dépôt de rapports, notation, soutenances) et un accès aux opportunités de stage publiées par les partenaires de l'ISITCOM, le tout dans une plateforme dédiée et adaptée aux besoins spécifiques de notre établissement.

1.6 Solution Proposée

Suite à l'analyse comparative des solutions existantes, nous proposons **WinLkar**, une solution en deux composantes complémentaires :

Composante 1 — Application Android native (Java) utilisant Firebase Realtime Database pour le partage de position GPS en temps réel. Cette composante couvre les interfaces passager (carte interactive, liste des stations, ETA), conducteur (sélection d'itinéraire, diffusion GPS, gestion du trajet) et administrateur (tableau de bord, gestion de la flotte).

Composante 2 — Backend avec Google Cloud Platform (Google Maps API + Google Firebase) offrant une API REST complète avec authentification JWT, gestion des rôles, système de réclamations, notifications et statistiques.

1.7 Environnement et Outils de Développement

1.7.1 Android Studio

Android Studio est l'environnement de développement intégré (IDE) officiel pour le développement d'applications Android, basé sur IntelliJ IDEA. Il a été utilisé pour écrire le code Java de l'application mobile, concevoir les interfaces XML, tester sur émulateur et appareil physique, et déboguer les composants Android (FusedLocationProviderClient, Firebase SDK, Google Maps SDK).

1.7.2 Adobe Photoshop

Adobe Photoshop a été utilisé pour la conception visuelle du projet : création du logo WinLkar, design des icônes personnalisées des marqueurs de bus et d'arrêts sur la carte, maquettes des écrans (mockups) avant implémentation.

1.7.3 GitHub et Git

Git est le système de contrôle de version distribué utilisé pour suivre toutes les modifications du code source. GitHub a servi de plateforme d'hébergement centralisée, permettant à l'équipe de six développeurs de travailler en parallèle via un modèle de branches (une branche par fonctionnalité ou sprint).

1.7.4 Firebase

Firebase est la plateforme de développement mobile de Google utilisée pour plusieurs services dans WinLkar :

— **Firebase Realtime Database** : base de données NoSQL synchronisée en temps réel, utilisée pour stocker et diffuser les positions GPS des bus actifs avec une latence minimale (inférieure à 200ms).

— **Firebase Authentication** : infrastructure d'authentification sécurisée pour les conducteurs et administrateurs.

— **Firebase Storage** : stockage des ressources statiques de l'application.

La structure `activeTrips/{busId}` permet une mise à jour instantanée visible par tous les passagers connectés en temps réel.

1.7.5 Google Maps Platform

La Google Maps Platform a été intégrée via deux SDK :

— **Maps SDK for Android** : affichage de la carte interactive, placement des marqueurs de bus et de stations, gestion des InfoWindows et des interactions utilisateur.

— **Places API** : autocomplétion des champs de recherche de lieux pour la sélection des points de départ et d'arrivée par le conducteur.

1.7.6 Tableau récapitulatif des outils

Table 1.2 – Outils et technologies utilisés dans le projet WinLkar

Outil	Catégorie	Utilisation dans WinLkar
Android Studio	IDE	Développement, test et débogage de l'application Android
Java (SDK 21)	Langage	Langage de programmation principal de l'app mobile
Adobe Photoshop	Design	Logo, icônes, maquettes UI
Git / GitHub	Versionnement	Suivi des modifications, branches de développement
Firebase Realtime DB	Base de données temps réel	Synchronisation GPS des bus actifs
Firebase Authentication	Authentification	Comptes conducteurs et administrateurs
Firebase Storage	Stockage	Ressources statiques
Google Maps SDK	Cartographie	Carte interactive, marqueurs, InfoWindows
Google Places API	Géolocalisation	Autocomplétion des points de trajet
Draw.io et StarUML	Modélisation	Diagrammes UML

1.8 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet WinLkar, la problématique rencontrée dans le suivi des transports en commun à Sousse, les objectifs du projet, une comparaison avec les solutions existantes, et les outils de développement utilisés. WinLkar se positionne comme une solution unique, combinant suivi temps réel, interface conducteur dédiée et panneau d'administration, spécifiquement conçue pour le contexte tunisien. Le chapitre suivant détaillera la conception et la spécification de la solution.

Chapitre 2

Conception et spécification du projet

2.1 Introduction

Ce chapitre présente la phase de Sprint 0 du projet WinLkar : organisation de l'équipe, architecture technique, modélisation des données, besoins fonctionnels et non fonctionnels, Product Backlog et les diagrammes UML (cas d'utilisation, classes, séquence).

2.2 Organisation de l'Équipe

2.2.1 Composition et rôles

L'équipe de développement WinLkar est composée de six membres ayant chacun des rôles bien définis dans le cadre de la méthode Scrum :

Table 2.1 – Composition de l'équipe WinLkar

Rôle Scrum	**Membre**
Scrum Master	Sadok Belhadj
Product Owner	Administration ISITCOM
Développeur Backend (Google Cloud Platform)	Alaa Eddine Kerkeni
Développeur Android	Ahmed Neji
Développeur Android / UI	Abdelaziz Youssef
Développeur Full Stack	Mohamed Karmous
Développeur Android / Firebase	Mohamed Salah Mejri

2.2.2 Outils de communication et de collaboration

Pour assurer une collaboration efficace entre les membres de l'équipe, les outils suivants ont été adoptés dès le Sprint 0 :

— **Discord** :

canal de communication principal pour les échanges quotidiens, les réunions de planification et les revues de sprint à distance ;

— **GitHub** :

plateforme de versionnement du code source et de gestion des branches de développement ;

— **Google Docs** :

outil de rédaction collaborative pour les documents de spécifications et les comptes-rendus de réunion ;

— **Draw.io** :

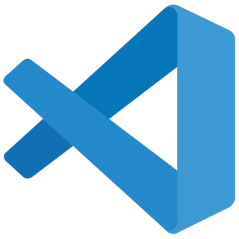
outil de modélisation UML pour la conception des diagrammes de cas d'utilisation et de classes.

2.3 Mise en place de l'environnement de développement

2.3.1 Installation et configuration des outils

Durant le Sprint 0, chaque membre de l'équipe a procédé à l'installation et à la configuration de l'environnement de développement local, afin de garantir une homogénéité des conditions de travail au sein de l'équipe. Les outils installés sont les suivants :

— **Visual Studio Code** :

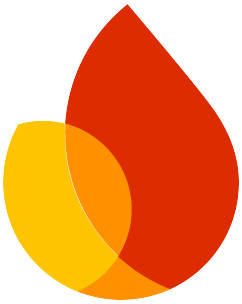


éditeur de code principal, avec les extensions adaptées au développement web (HTML, CSS, JavaScript) ;

— **Git** :

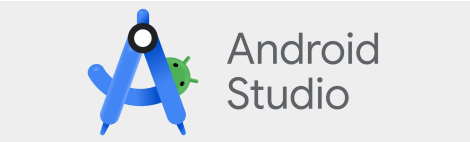
outil de versionnement du code source, configuré avec les identifiants GitHub de chaque développeur ;

— **Firebase CLI** :



interface en ligne de commande pour la configuration et le déploiement de la base de données Firebase ;

— **Android Studio** :



Android Studio, utilisé pour tester les interfaces développées.

Inscrive.io :

Inscrive.io est un éditeur LaTeX en ligne gratuit permettant de rédiger et de compiler des documents LaTeX directement depuis le navigateur, sans installation préalable. Il offre une interface intuitive avec une prévisualisation en temps réel du document compilé. Nous avons utilisé Inscrive.io pour la rédaction et la mise en forme de notre rapport de fin d'année, en remplacement d'Overleaf.

Adobe Photoshop :

Adobe Photoshop est un logiciel professionnel de retouche et de création graphique développé par Adobe. Il permet de créer et de modifier des images, des logos et des icônes avec une grande précision. Dans le cadre de notre projet, nous avons utilisé Photoshop pour concevoir le logo de la plateforme ainsi que les différentes icônes utilisées dans l'interface utilisateur.

2.4 Architecture Technique

2.4.1 Vue d'ensemble de l'architecture

L'architecture de WinLkar est une architecture à trois tiers :

— **Tiers présentation** : Application Android native (Java), interface utilisateur passager, conducteur et administrateur.

— **Tiers logique** : Firebase Realtime Database (temps réel) + API Google Maps(Directions + Places API) pour la logique métier.

— **Tiers données** : Firebase Realtime Database.

2.4.2 Stack technologique

Table 2.2 – Stack technologique de WinLkar

Couche	**Technologie**	**Rôle**
Mobile	Android (Java)	Interface utilisateur passager, conducteur, admin
Cartographie	Google Maps SDK	Affichage des bus et stations sur carte interactive
Base de données temps réel	Firebase Realtime DB	Synchronisation GPS des positions GPS en direct
Authentification	Firebase Authentication	Sécurisation des comptes conducteurs et admin
Backend	Directions + Places	Google Maps API : utilisateurs, trajets, réclamations
Persistance	Google Firebase	Stockage des entités métier
Cache	Google Firebase	Requêtes géospatiales fréquentes
Temps réel backend	Google Firebase	Notifications push
Déploiement	Google Cloud Platforme	Orchestration des microservices
Versionnement	Git / GitHub	Suivi des modifications

2.5 Diagramme de Cas d'Utilisation Global

Le diagramme de cas d'utilisation ci-dessous représente l'ensemble des interactions entre les trois acteurs principaux (Passager, Conducteur, Administrateur) et le système WinLkar.

FIGURE 2.1 – Diagramme de cas d'utilisation global — WinLkar

2.6 Diagramme de Classes

Le diagramme de classes représente les principales entités du système et leurs relations.

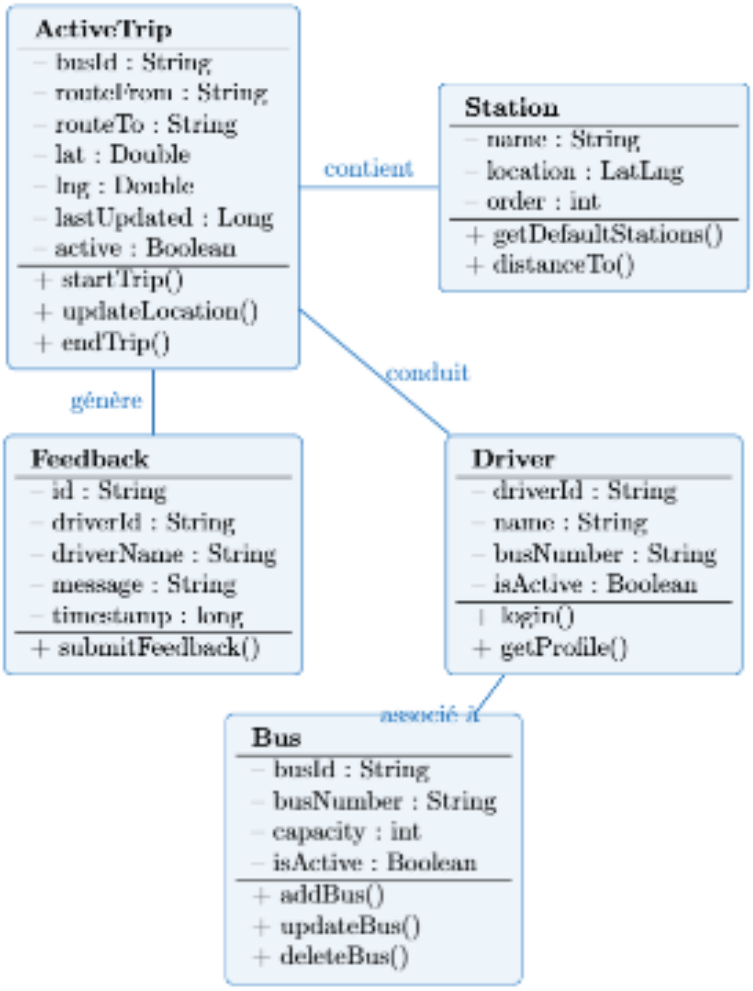


FIGURE 2.2 – Diagramme de cas d'utilisation global — WinLkar

2.7 Diagramme de Séquence — Authentification

Ce diagramme décrit le processus d'authentification du conducteur via Firebase Authentication.

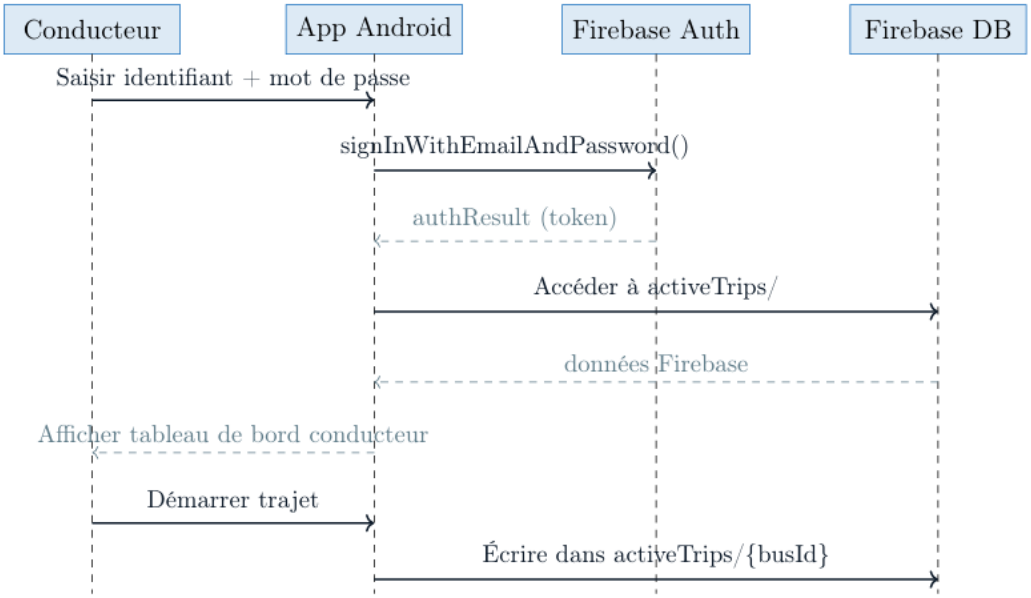


FIGURE 2.3 – Diagramme de séquence — Authentification du conducteur

2.8 Diagramme de Séquence — Mise à jour GPS

Ce diagramme illustre le flux de mise à jour de position GPS du conducteur vers les passagers.

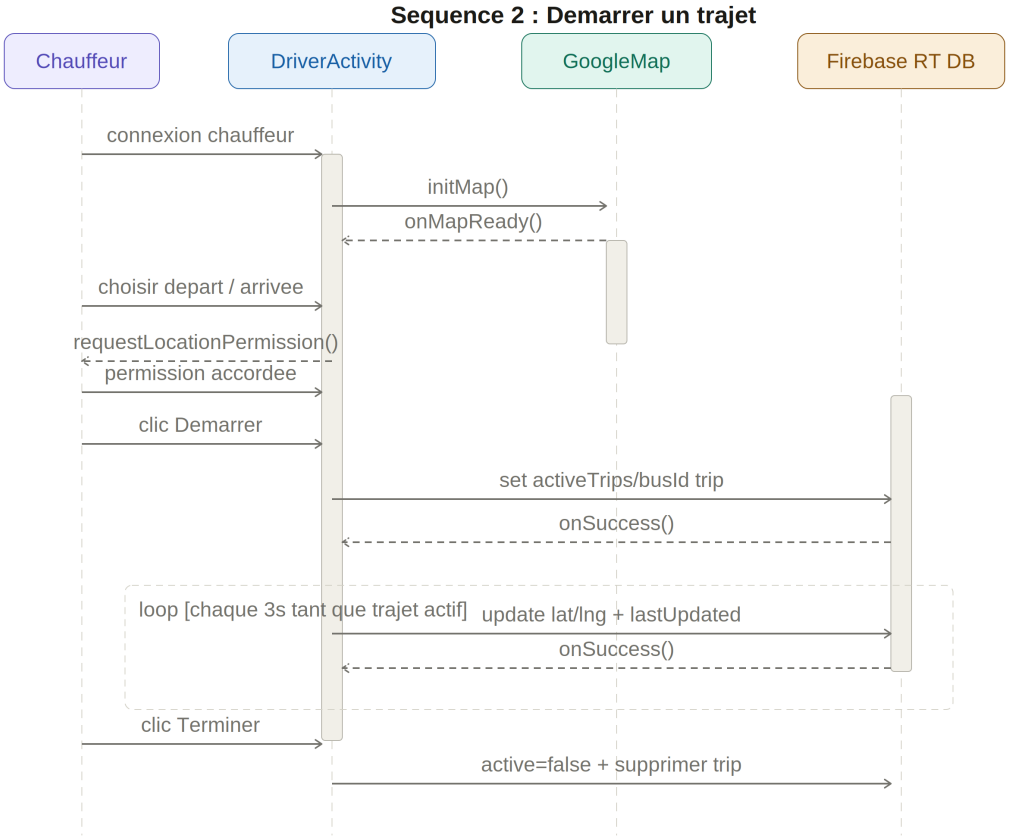


FIGURE 2.4 – Diagramme de séquence — Mise à jour GPS temps réel

2.9 Modélisation des Données

2.9.1 Entités principales

Le modèle de données MongoDB comprend les entités suivantes :

- **User** : uid, nom, email, mot de passe hashé (bcrypt), rôle (passenger | driver | admin), date de création.
- **Bus** : busId, numéro, capacité, statut actif.
- **Driver** : driverId, nom, busNumber, statut actif, référence vers User.
- **ActiveTrip** (Firebase) : busId, routeFrom, routeTo, routeDescription, lat, lng, lastUpdated, active.
- **Station** : nom, latitude, longitude, ordre dans le trajet.
- **Feedback** : id, driverId, driverName, message, timestamp.
- **TripRating** : ratingId, tripId, passengerName, note (1-5), commentaire.
- **Complaint** : complaintId, type, sujet, message, priorité, statut, réponse.
- **Notification** : notification système pour les utilisateurs.

2.9.2 Schéma Firebase Realtime Database

La structure Firebase est organisée autour de nœuds clés. Le nœud principal `activeTrips` contient un sous-nœud par bus identifié par son `busId` (par exemple `BUS-001`). Chaque

entrée stocke les informations du trajet en cours : le point de départ (`routeFrom`), la destination (`routeTo`), une description textuelle de l'itinéraire (`routeDescription`), la latitude et la longitude courantes du bus, un timestamp Unix de la dernière mise à jour GPS (`lastUpdated`), et un booléen `active` indiquant si le trajet est en cours. Les autres nœuds racine — `buses`, `drivers` et `feedbacks` — stockent respectivement les informations de la flotte, les comptes des conducteurs et les retours soumis par les passagers.

2.10 Besoins Fonctionnels

2.10.1 Besoins du Passager

- Visualiser en temps réel tous les bus actifs sur une carte Google Maps.
- Consulter les détails d'un bus (ID, description du trajet, ETA).
- Voir la liste de toutes les stations disponibles dans la région de Sousse.
- Rechercher une station par nom.
- Trouver les bus les plus proches de sa position géographique.
- Ajouter et gérer des trajets favoris.
- Consulter l'historique de ses trajets.
- Noter un trajet (1 à 5 étoiles) et laisser un commentaire.
- Soumettre une réclamation concernant un trajet.
- Recevoir et consulter des notifications.

2.10.2 Besoins du Conducteur

- Se connecter avec son compte (rôle driver) via Firebase Authentication.
- Sélectionner les points de départ et d'arrivée via Google Places Autocomplete.
- Générer ou saisir un identifiant de bus unique.
- Démarrer un trajet et partager sa position GPS en temps réel.
- Mettre à jour la station courante et la prochaine station.
- Terminer un trajet et retirer le bus de la liste active.

2.10.3 Besoins de l'Administrateur

- Gérer les comptes des conducteurs (ajout, modification, suppression).
- Gérer la flotte de bus (ajout, modification, suppression).
- Visualiser les trajets actifs en temps réel.
- Consulter et traiter les feedbacks et réclamations.
- Accéder aux statistiques du système.

2.11 Besoins Non Fonctionnels

— **Performance** : la mise à jour de la position GPS doit être visible par les passagers en moins de 500ms. Cache de Google Firebase pour les requêtes géospatiales fréquentes.

— **Sécurité** : mots de passe hashés avec bcrypt (12 rounds). Authentification de Firebase DB avec expiration. Firebase Security Rules pour protéger les données sensibles.

— **Scalabilité** : architecture de Google Cloud Platform permettant de passer à une architecture microservices.

— **Disponibilité** : Firebase assure la synchronisation même en cas d'interruption courte du réseau grâce au cache local.

— **Usabilité** : interface sombre et épurée adaptée à une utilisation en mobilité.

— **Documentation** : Google Maps API disponible à `/app/services`$_{j}sonpourtouslesendpointsRE$

2.12 Product Backlog

Sprint 1 — Fonctionnalités Core & Temps Réel

Table 2.3: Product Backlog — Sprint 1 (9 User Stories)

ID	Acteur	Description (En tant que... je veux...)	Priorité	Sprint	Statut
US01	Passager	Créer un compte pour accéder aux fonctionnalités personnalisées de l'application	Élevée	1	Terminé
US02	Passager	Me connecter à mon espace personnel avec mes identifiants	Élevée	1	Terminé
US03	Conducteur	Gérer mes trajets et partager ma position GPS en temps réel	Élevée	1	Terminé
US04	Conducteur	Sélectionner un itinéraire (départ/arrivée) via Google Places Autocomplete	Élevée	1	Terminé
US05	Conducteur	Partager ma position GPS en continu toutes les 3 secondes	Élevée	1	Terminé
US06	Passager	Voir tous les bus actifs sur une carte Google Maps interactive	Élevée	1	Terminé
US07	Passager	Voir l'ETA estimé vers chaque arrêt calculé avec la formule de Haversine	Élevée	1	Terminé
US08	Admin	Gérer les utilisateurs (conducteurs, bus) depuis le tableau de bord	Élevée	1	Terminé
US09	Conducteur	Terminer un trajet et retirer automatiquement le bus de la liste active	Moyenne	1	Terminé

Sprint 2 — Services Backend & Interaction Utilisateur

Table 2.4: Product Backlog — Sprint 2 (7 User Stories)

ID	Acteur	Description (En tant que... je veux...)	Priorité	Sprint	Statut
US10	Passager	Trouver les bus les plus proches de ma position géographique	Moyenne	2	Terminé
US11	Passager	Ajouter un trajet en favori pour le retrouver rapidement	Moyenne	2	Terminé
US12	Passager	Consulter l'historique de mes trajets précédents	Moyenne	2	Terminé
US13	Passager	Noter un trajet de 1 à 5 étoiles et laisser un commentaire	Moyenne	2	Terminé
US14	Passager	Soumettre une réclamation concernant un trajet ou un conducteur	Moyenne	2	Terminé
US15	Admin	Gérer la flotte de bus (ajout, modification, suppression)	Moyenne	2	Terminé
US16	Admin	Traiter les réclamations des passagers et envoyer des réponses	Faible	2	Terminé

2.13 Méthodologie de Développement — Scrum

La réussite de WinLkar repose autant sur la méthode de travail adoptée que sur les compétences techniques. Nous avons choisi la méthode agile **Scrum**, particulièrement adaptée aux projets de développement logiciel impliquant une équipe de taille moyenne et des exigences susceptibles d'évoluer.

2.13.1 Les rôles Scrum dans WinLkar

- **Product Owner (Administration ISITCOM)** : définit les priorités du Product Backlog et valide les fonctionnalités livrées.
- **Scrum Master (Sadok Belhadj)** : facilite le processus Scrum, élimine les obstacles et anime les cérémonies.
- **Équipe de développement (4 membres)** : auto-organisée, responsable de la conception, réalisation et tests.

2.13.2 Planification des Sprints

Table 2.5 – Planification des sprints WinLkar

Sprint	Durée	Contenu principal
Sprint 0	1 semaine	Initialisation, architecture, configuration Firebase et Android Studio, Product Backlog
Sprint 1	3 semaines	Authentification, interface Android (passager/conducteur), Firebase Realtime DB, Google Maps
Sprint 2	3 semaines	Backend Google Cloud Platform, Google Maps API, réclamations, notation, historique

2.14 Conclusion

Ce chapitre a présenté la conception complète du projet WinLkar : diagrammes UML, modélisation des données, besoins fonctionnels et non fonctionnels, Product Backlog de 16 user stories, et la méthodologie Scrum adoptée. Les deux sprints de développement sont détaillés dans les chapitres suivants.

Chapitre 3

Réalisation — Sprint 1

3.1 Introduction

Ce chapitre présente les fonctionnalités développées durant le Sprint 1, qui constitue le cœur de l'application WinLkar : l'interface passager avec suivi temps réel des bus et l'interface conducteur avec partage de position GPS via Firebase Realtime Database.

L'objectif principal du Sprint 1 était de développer les fonctionnalités core de l'application Android permettant le suivi en temps réel des bus par les passagers et la déclaration de trajet par les conducteurs via Firebase Realtime Database, le tout sans backend supplémentaire dans un premier temps.

3.2 Backlog du Sprint 1

Le Sprint 1 couvre les 9 premières user stories du Product Backlog (US01 à US09), axées sur les fonctionnalités core Android et Firebase.

Table 3.1 – Product Backlog — Sprint 1

ID	Acteur	Description	Priorité	Statut
US01	Passager	Créer un compte pour accéder aux fonctionnalités personnalisées	Élevée	Terminé
US02	Passager	Se connecter à mon espace personnel avec mes identifiants	Élevée	Terminé
US03	Conducteur	Gérer mes trajets et partager ma position GPS en temps réel	Élevée	Terminé
US04	Conducteur	Sélectionner un itinéraire via Google Places Autocomplete	Élevée	Terminé
US05	Conducteur	Partager ma position GPS en continu toutes les 3 secondes	Élevée	Terminé
US06	Passager	Voir tous les bus actifs sur une carte Google Maps interactive	Élevée	Terminé
US07	Passager	Voir l'ETA estimé vers chaque arrêt (formule de Haversine)	Élevée	Terminé
US08	Admin	Gérer les utilisateurs (conducteurs, bus) depuis le tableau de bord	Élevée	Terminé
US09	Conducteur	Terminer un trajet et retirer automatiquement le bus de la liste active	Moyenne	Terminé

3.3 Conception

3.3.1 Diagramme de Cas d'Utilisation — Sprint 1

Ce diagramme détaille les cas d'utilisation implémentés lors du Sprint 1, correspondant aux fonctionnalités cœur de l'application Android.

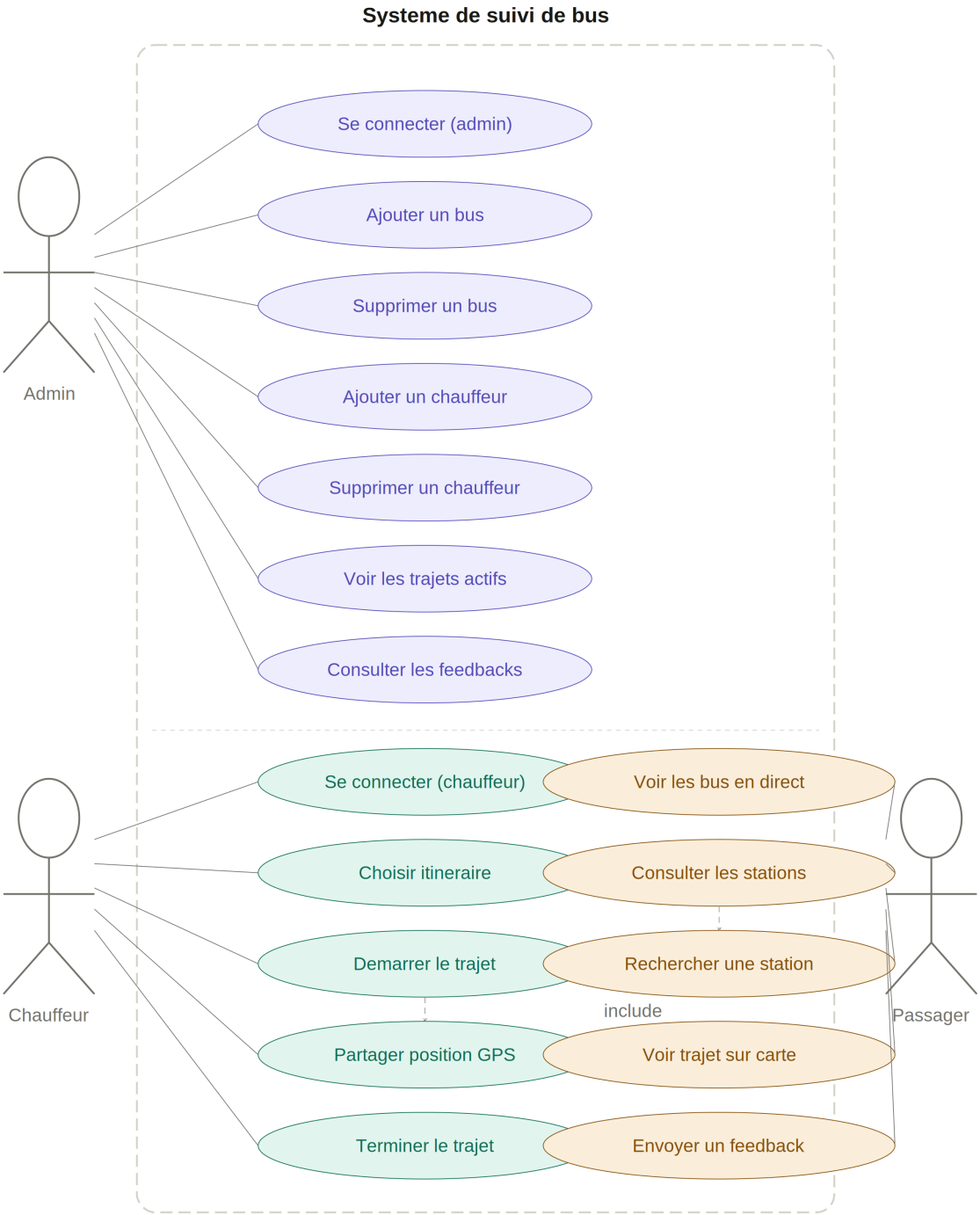


FIGURE 3.1 – Diagramme de cas d'utilisations - Sprint 1

3.3.2 Diagrammes de Séquence — Sprint 1

Les diagrammes de séquence ci-dessous illustrent les principaux flux du Sprint 1 : l'authentification de l'administrateur, le démarrage d'un trajet par le conducteur, le suivi en direct côté passager, et l'envoi de feedback.

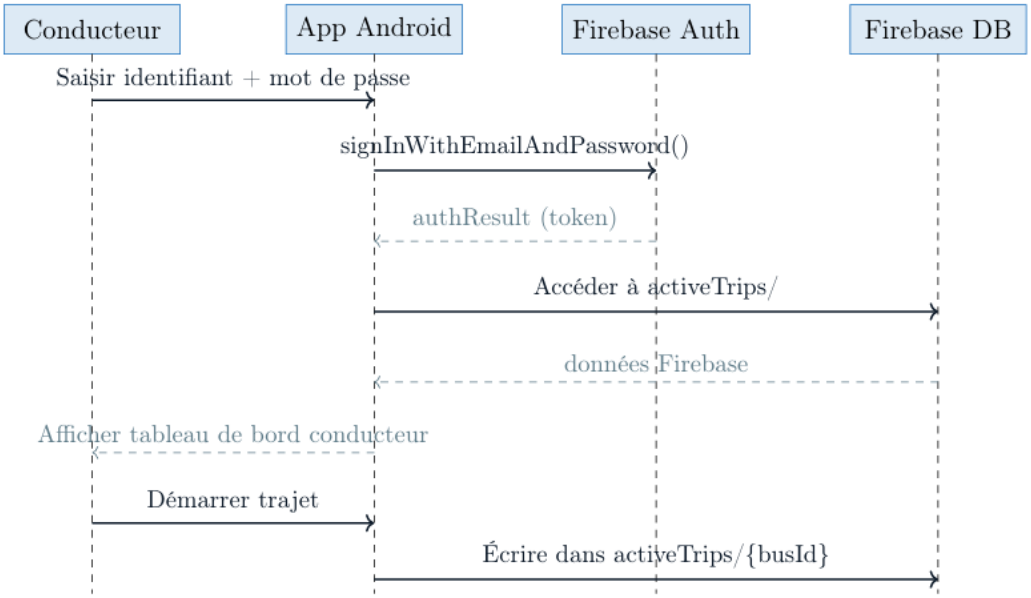


FIGURE 3.2 – Diagramme de séquence — Connexion Admin

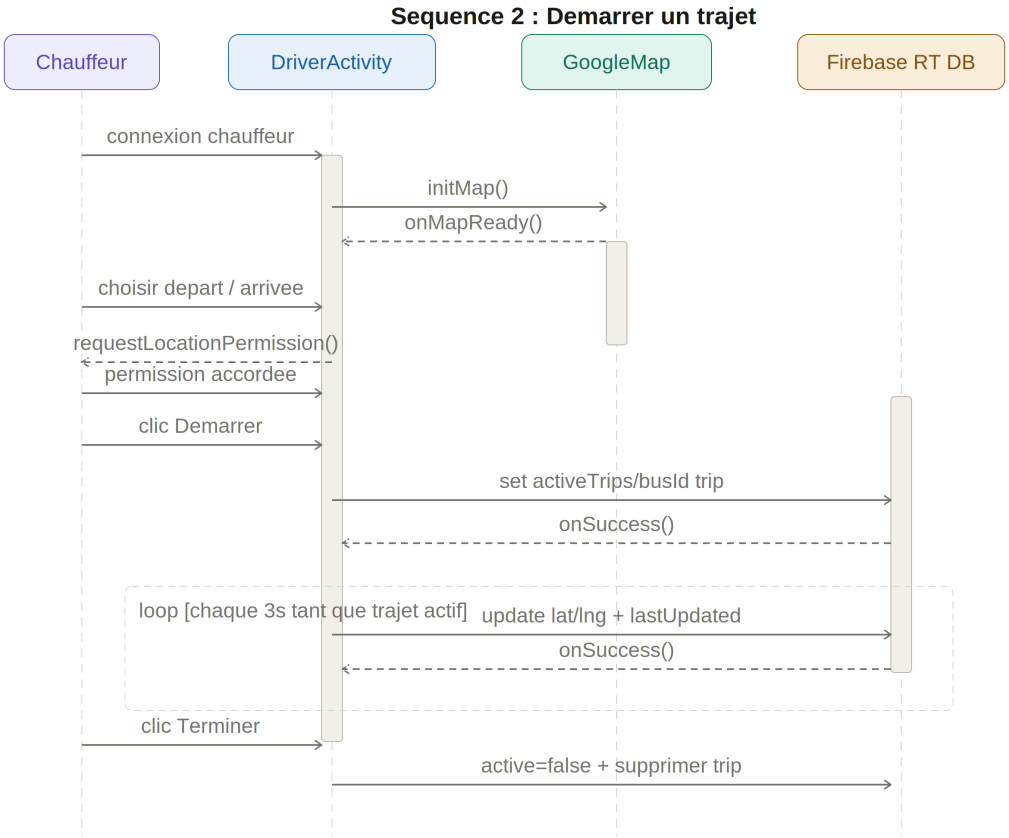


FIGURE 3.3 – Diagramme de séquence — Démarrer un trajet (Conducteur)

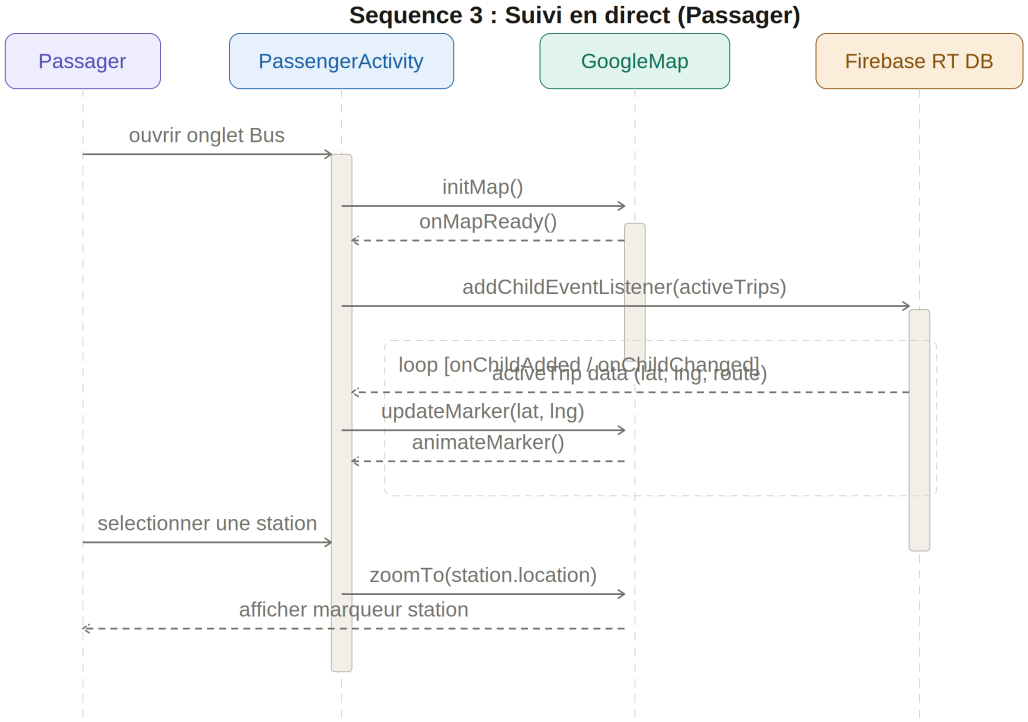


FIGURE 3.4 – Diagramme de séquence — Suivi en direct (Passager)

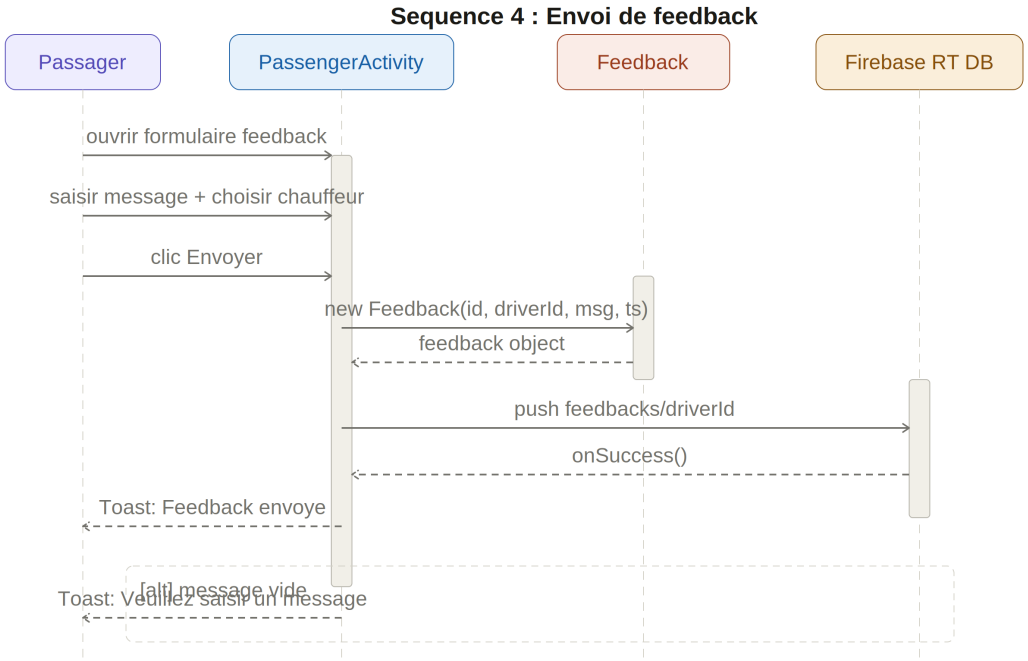


FIGURE 3.5 – Diagramme de séquence — Envoi de feedback

3.4 Réalisation

L'interface WinLkar adopte un design sombre (dark theme) avec une palette de couleurs bleue et cyan, offrant une lisibilité optimale en toutes conditions, notamment en extérieur sous le soleil.

L'écran d'accueil permet à l'utilisateur de choisir son rôle parmi trois options : **Passager**, **Conducteur** et **Administrateur**. Chaque option est représentée par une icône distinctive accompagnée d'une courte description de ses fonctionnalités. Un badge **LIVE** affiché dans le coin supérieur droit indique que le système est connecté à Firebase et diffuse des données en temps réel. L'écran de connexion du conducteur présente un formulaire sécurisé composé d'un champ identifiant et d'un champ mot de passe avec option de visibilité. Le portail administrateur dispose d'un écran dédié demandant un Admin ID et un mot de passe, avec un bouton *Authorize Access* qui vérifie les identifiants via Firebase Authentication avant d'ouvrir le tableau de bord.

3.5 Interface Passager

3.5.1 Liste des Stations

L'écran **All Stations** affiche la liste complète des stations disponibles dans la région de Sousse. Chaque station est présentée avec son nom et le statut des bus actifs (*No active buses* lorsqu'aucun bus n'est en service). Une barre de recherche en haut de l'écran permet de filtrer rapidement les stations par nom. Un bouton *Open Map* permet d'accéder directement à la carte interactive centrée sur la station sélectionnée.

Les stations implémentées couvrent l'ensemble de la région côtière de Sousse : Sousse Centrale (35.8282°N, 10.6358°E), Trocadero, Sahloul, Sousse Riadh, Hammam Sousse, Port El Kantaoui, Akouda, Kalaa Sghira, M'saken et Monastir Airport.

3.5.2 Carte interactive et suivi en temps réel

L'interface passager (PassengerActivity) est le cœur de l'application. Elle affiche une carte Google Maps interactive avec :

- Les marqueurs de tous les bus actifs, mis à jour en temps réel via Firebase.
- Les marqueurs des stations prédéfinies.
- Un InfoWindow s'affichant au clic sur un bus, montrant l'ID du bus, l'itinéraire et l'ETA.
- Un tracé de polyline indiquant l'itinéraire du bus sélectionné.
- Une animation de déplacement des marqueurs de bus entre deux positions GPS.

3.6 Code Source Java — Extraits Essentiels

Cette section présente les deux extraits de code les plus représentatifs de l'implémentation Android.

3.6.1 PassengerActivity.java — Listener Firebase temps réel

L'interface passager utilise un `ChildEventListener` pour réagir immédiatement à l'ajout, la modification ou la suppression de bus actifs dans Firebase. La méthode `setupFirebaseListener()` obtient une référence vers le nœud `activeTrips` dans Firebase Realtime Database. Trois callbacks sont implémentés : `onChildAdded` qui détecte l'apparition d'un nouveau bus actif et appelle `addOrUpdateBusMarker()` pour l'afficher sur la carte, `onChildChanged` qui met à jour la position du marqueur dès que les coordonnées GPS changent, et `onChildRemoved` qui supprime le marqueur de la carte et retire le bus de la liste locale lorsque le conducteur termine son trajet. Ce mécanisme garantit une synchronisation en temps réel sans aucune requête de polling.

3.6.2 Calcul de l'ETA — Formule de Haversine

L'ETA (Estimated Time of Arrival) est calculé côté client à partir de la formule de Haversine et d'une vitesse moyenne de 25 km/h. La méthode `calculateETA()` reçoit la position courante du bus et la position de la station cible, toutes deux exprimées en latitude/longitude. Elle convertit les coordonnées en radians, calcule les différences angulaires, puis applique la formule de Haversine pour obtenir la distance en kilomètres sur la surface sphérique de la Terre (rayon de 6 371 km). Le temps d'arrivée estimé est ensuite déduit en divisant cette distance par la vitesse moyenne de 25 km/h et en convertissant le résultat en minutes.

3.7 Interface Conducteur

3.7.1 Fonctionnalités implémentées

L'interface conducteur (DriverActivity) offre les fonctionnalités suivantes :

- **Sélection du départ et de l'arrivée** via Google Places Autocomplete.
- **Saisie ou génération automatique** d'un identifiant de bus (UUID).
- **Démarrage du trajet** : écriture dans Firebase sous `activeTrips/{busId}`.
- **Publication continue de la position GPS** via FusedLocationProviderClient, toutes les 3 secondes.
- **Affichage du marqueur bus** sur la carte avec mise à jour en temps réel.
- **Fin du trajet** : suppression de l'entrée Firebase et arrêt du service GPS.

3.8 Calcul de l'ETA

L'ETA est calculé côté client à partir de la formule de Haversine pour la distance et d'une vitesse moyenne de 25 km/h. La méthode convertit les coordonnées GPS du bus et de la station en radians, calcule les différences de latitude et de longitude, puis applique la formule afin d'obtenir la distance réelle en kilomètres sur la surface terrestre. Le résultat est ensuite converti en minutes en le divisant par la vitesse moyenne de déplacement du bus.

3.9 Modèle de Données Firebase

Table 3.2 – Structure du nœud activeTrips dans Firebase

Champ	Type	Description
busId	String	Identifiant unique du bus (ex : BUS-UUID)
routeFrom	String	Point de départ du trajet
routeTo	String	Point d'arrivée du trajet
routeDescription	String	Description textuelle de l'itinéraire
lat	Double	Latitude courante du bus
lng	Double	Longitude courante du bus
lastUpdated	Long	Timestamp Unix de la dernière mise à jour GPS
active	Boolean	Indique si le bus est actuellement en service

3.10 Tests et Validation

Les tests du Sprint 1 ont été réalisés sur :

- **Émulateur Android** (API 34, Pixel 6) via Android Studio.
- **Appareil physique** (Samsung Galaxy A54, Android 14).
- **Simulation de position GPS** via l'émulateur pour tester le déplacement des marqueurs.
- **Tests multi-clients** : un conducteur sur appareil physique + un passager sur émulateur pour vérifier la synchronisation en temps réel.

3.11 Sprint 1 Review

Table 3.3 – Bilan du Sprint 1

Élément	Détail
User stories terminées	US01 – US09 (9/9)
Démo réalisée	Oui — sur émulateur et appareil physique
Retours équipe	Interface conducteur intuitive, ETA à affiner
Points à améliorer	Précision du calcul ETA, gestion erreurs réseau
Vélocité	34 story points complétés

3.12 Conclusion

Le Sprint 1 a permis de développer et valider les fonctionnalités essentielles de WinLkar : le suivi GPS temps réel via Firebase, la carte interactive passager, et l'interface conducteur. Toutes les 9 user stories du Sprint 1 ont été complétées. Le Sprint 2 étendra ces fonctionnalités avec un backend API complet.

Chapitre 4

Réalisation — Sprint 2

4.1 Introduction

Le Sprint 2 est consacré au développement du backend de Google Cloud et à l'intégration des fonctionnalités avancées : authentification sécurisée, gestion des conducteurs et des bus par l'administrateur, système de feedbacks, et architecture complète de l'API.

L'objectif du Sprint 2 était de développer le backend API Google Maps avec authentification de Google Firebase BD, gestion complète des entités (conducteurs, bus, trajets, réclamations), et d'intégrer l'interface administrateur Android avec Firebase.

4.2 Backlog du Sprint 2

Le Sprint 2 couvre les user stories US10 à US16, axées sur les services backend et les interactions utilisateur avancées.

Table 4.1 – Product Backlog — Sprint 2

ID	Acteur	Description	Priorité	Statut
US10	Passager	Trouver les bus les plus proches de ma position géographique	Moyenne	Terminé
US11	Passager	Ajouter un trajet en favori pour le retrouver rapidement	Moyenne	Terminé
US12	Passager	Consulter l'historique de mes trajets précédents	Moyenne	Terminé
US13	Passager	Noter un trajet de 1 à 5 étoiles et laisser un commentaire	Moyenne	Terminé
US14	Passager	Soumettre une réclamation concernant un trajet ou un conducteur	Moyenne	Terminé
US15	Admin	Gérer la flotte de bus (ajout, modification, suppression)	Moyenne	Terminé
US16	Admin	Traiter les réclamations des passagers et envoyer des réponses	Faible	Terminé

4.3 Conception

4.3.1 Diagramme de Cas d'Utilisation — Sprint 2

Ce diagramme illustre les cas d'utilisation spécifiques au Sprint 2, centré sur le backend API et les fonctionnalités avancées.

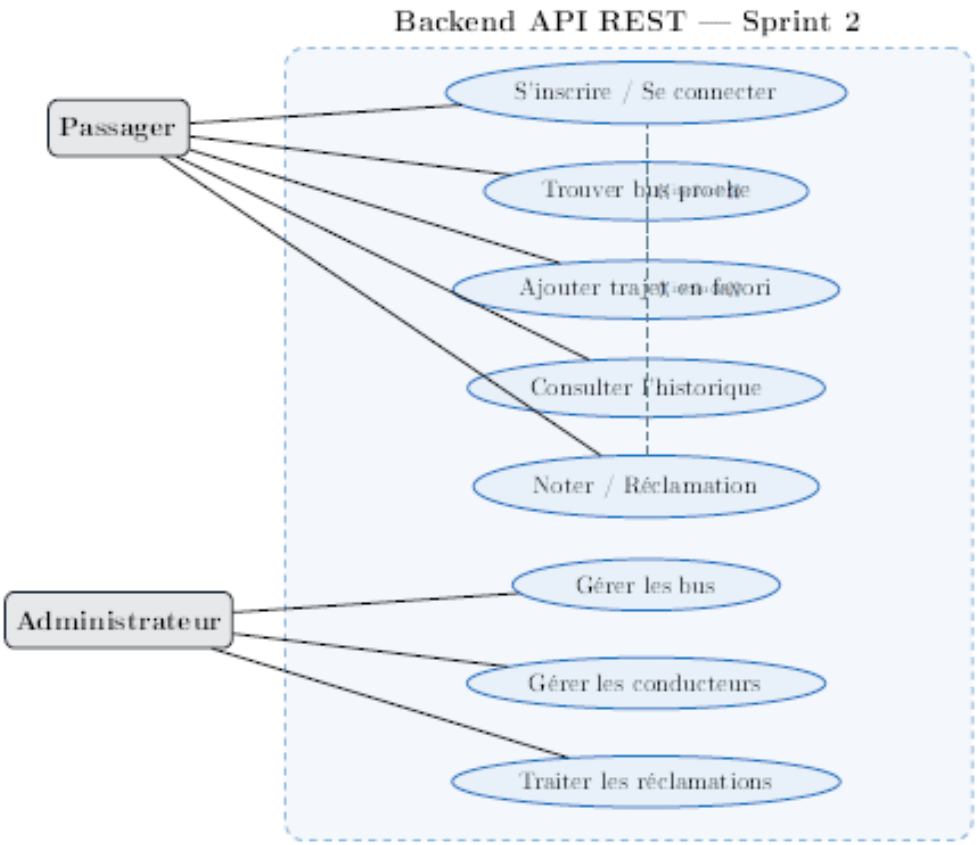


Figure 4.1: Diagramme de cas d'utilisation — Sprint 2

FIGURE 4.1 – Diagramme de cas d'utilisation — Sprint 2

4.3.2 Architecture Backend — Diagramme de Séquence JWT

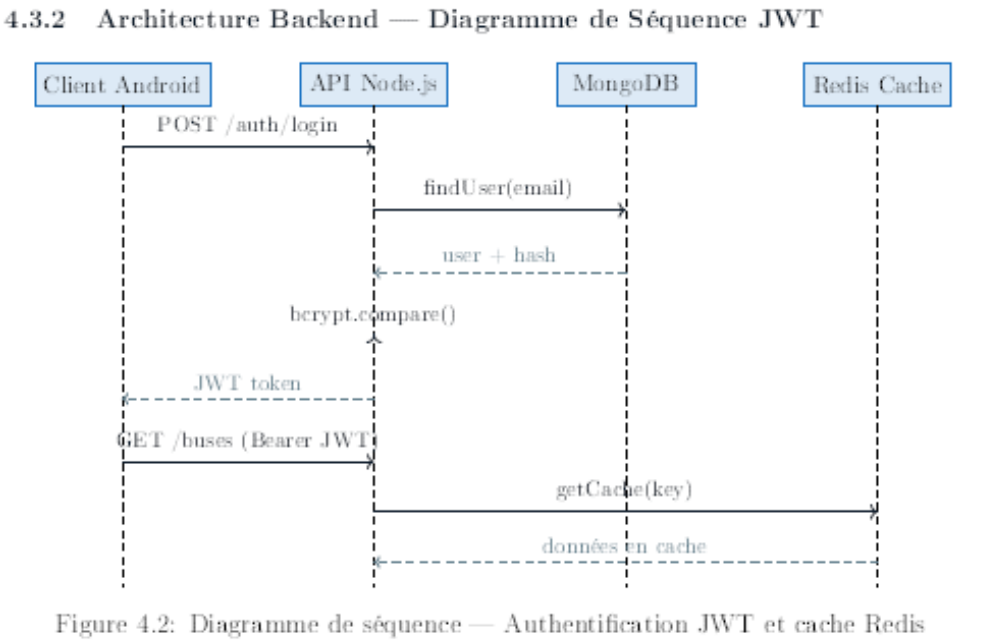


FIGURE 4.2 – Diagramme de séquence — Authentification Firebase

4.4 Réalisation

Le backend suit le pattern Model-View-Controller (MVC). Le projet est organisé en plusieurs dossiers distincts : `config/` regroupe les fichiers de connexion à Firebase DB,la configuration de google Cloud Platform. Le dossier `models/` contient les schémas Mongoose pour chaque entité : `User`, `Bus`, `Driver`, `Trip`, `TripRating` et `Complaint`. Les `controllers/` orchestrent la logique métier pour l'authentification, les trajets et les fonctions administrateur. Le dossier `services/` isole les services réutilisables tels que le calcul d'ETA et l'envoi de notifications. Les `routes/` définissent les points d'entrée HTTP de l'API, le dossier `middleware/` contient la vérification de BD, la gestion des rôles et le rate limiting, et `sockets/` gère les événements de GPS en temps réel.

4.4.1 Authentification et sécurité

L'authentification est implémentée avec les mécanismes suivants :

— **Java** (12 rounds) pour le hashage des mots de passe.

— **Tokens Firebase** contenant l'identifiant utilisateur et son rôle, avec expiration à 7 jours.

— **FIrebase** vérifiant le token Bearer à chaque requête protégée.

Le middleware d'authentification BD a été implémenté dans le fichier `middleware/jwt.js`. À chaque requête protégée, il extrait le token Bearer depuis l'en-tête `Authorization`.

Si l'en-tête est absent ou mal formé, une erreur 401 est immédiatement renvoyée au client. Dans le cas contraire, le token est vérifié à l'aide de la clé secrète `JWT_SECRET` stockée dans les variables d'environnement. En cas de succès, les informations décodées (identifiant utilisateur, rôle) sont attachées à l'objet requête `req.user` et transmises au contrôleur suivant. Si le token est invalide ou expiré, une erreur 401 est renvoyée.

4.5 Interface Administrateur

4.5.1 Fonctionnalités de l'AdminActivity

L'interface administrateur (AdminActivity) offre un tableau de bord complet directement dans l'application Android. Les fonctionnalités implémentées sont :

— **Gestion des trajets actifs** : liste en temps réel de tous les bus en service, avec possibilité de les supprimer manuellement.

— **Gestion des bus** : ajout, modification et suppression de bus de la flotte.

— **Gestion des conducteurs** : ajout, modification et suppression de comptes conducteurs.

— **Consultation des feedbacks** : liste de tous les feedbacks soumis par les conducteurs.

4.5.2 Portail d'accès administrateur

L'accès au portail administrateur est sécurisé par un écran de connexion dédié qui demande un Admin ID et un mot de passe. Lors de la soumission, l'application vérifie les identifiants via Firebase Authentication : si les informations sont correctes, l'utilisateur est redirigé vers le tableau de bord administrateur où les méthodes `loadBuses()` et `loadDrivers()` sont immédiatement appelées pour charger les données depuis Firebase. En cas d'échec, un message *Toast* signale des identifiants invalides.

4.6 Structure de la Base de Données Firebase

Table 4.2 – Structure complète des données Firebase

Nœud	Accès	Description
activeTrips/	Public	Positions GPS temps réel des bus actifs
buses/	Admin	Informations sur les bus de la flotte
drivers/	Admin	Comptes des conducteurs
stations/	Public	Liste des arrêts de bus
feedbacks/	Public	Feedbacks soumis par les conducteurs
ratings/	Public	Notes et commentaires des passagers
complaints/	Admin	Réclamations des passagers

4.7 Service de Géolocalisation

Le service de géolocalisation backend permet de trouver les bus actifs les plus proches d'une position donnée :

— Utilisation de l'index MongoDB `2dsphere` pour les requêtes géospatiales.

— Opérateur MongoDB `$near` pour rechercher les bus dans un rayon configurable.

— Calcul de distance client-serveur avec la **formule de Haversine**.

— Cache **Redis** pendant 30 secondes pour les requêtes géospatiales fréquentes.

— Tri des résultats par distance croissante.

4.8 Service ETA Backend

Le service ETA backend, implémenté dans `services/etaService.js`, calcule le temps d'arrivée estimé pour chaque station sur l'itinéraire d'un bus. Il définit d'abord une fonction `haversineDistance()` qui calcule la distance en kilomètres entre deux points GPS en appliquant la formule de Haversine sur un rayon terrestre de 6 371 km. La fonction principale `calculateETAs()` reçoit la position actuelle du bus et la liste des stations, les trie par ordre d'arrêt, puis pour chaque station calcule la distance et en déduit l'ETA en minutes en supposant une vitesse moyenne de 25 km/h. Elle retourne un tableau enrichi comprenant la distance en kilomètres et l'ETA arrondi à la minute pour chaque arrêt.

4.9 API REST — Endpoints principaux

Table 4.3 – Principaux endpoints de l'API REST WinLkar

Méthode	**Endpoint**	**Auth**	**Description**
POST	/api/auth/login	Non	Connexion utilisateur
POST	/api/auth/register	Non	Création de compte
GET	/api/trips/active	Non	Liste des trajets actifs
GET	/api/trips/nearby	Non	Bus à proximité
POST	/api/trips/start	Driver	Démarrer un trajet
PUT	/api/trips/ :id/location	Driver	Mettre à jour la position
POST	/api/trips/ :id/end	Driver	Terminer un trajet
GET	/api/buses	Admin	Liste des bus
POST	/api/buses	Admin	Ajouter un bus
DELETE	/api/buses/ :id	Admin	Supprimer un bus
GET	/api/drivers	Admin	Liste des conducteurs
POST	/api/complaints	User	Soumettre une réclamation
GET	/api/complaints	Admin	Liste des réclamations
POST	/api/ratings	User	Noter un trajet
GET	/api/stations	Non	Liste des stations

4.10 Déploiement avec Google Cloud Platforme

Le backend est déployable via Google Cloud Platform avec trois services orchestrés. Le service principal `app` construit l'image Gradle à partir du `Android Studio` local, expose le port 3000, et reçoit les variables d'environnement nécessaires : l'URI de connexion Firebase, l'URL Redis et la clé secrète BD. Il dépend des services `mongo` et `redis`, qui démarrent avant lui. Le service `mongo` utilise l'image officielle Firebase et monte un volume persistant pour conserver les données entre les redémarrages. Le service `redis` repose sur l'image officielle Alpine, légère et optimisée pour la production.

4.11 Tests et Documentation

4.11.1 Documentation de Google Cloud

La documentation API est automatiquement générée et disponible à l'URL `https://cloud.googl` Elle permet aux développeurs front-end de tester les endpoints directement depuis le navigateur.

4.12 Sprint 2 Review

Table 4.4 – Bilan du Sprint 2

Élément	**Détail**
User stories terminées	US10 – US16 (7/7)
Tests API	Tous les endpoints testés avec Google Cloud
Documentation	Google Maps disponible à /api/docs
Déploiement	BD fonctionnel (Firebase + Google Cloud Platform)
Retours équipe	API complète et documentée
Vélocité	28 story points complétés
Total projet	16/16 user stories, 62 story points

4.13 Conclusion

Le Sprint 2 a complété l'architecture backend de WinLkar. L'ensemble des 16 user stories définies dans le Product Backlog ont été réalisées avec succès sur les deux sprints, pour un total de 62 story points. L'application est désormais fonctionnelle de bout en bout.

Conclusion Générale

Ce projet de fin d'année nous a permis de concevoir et développer **WinLkar**, une application mobile Android complète de suivi de bus en temps réel, adaptée au contexte tunisien et particulièrement à la région de Sousse.

Bilan technique

Sur le plan technique, ce projet nous a permis de maîtriser :

- Le **développement Android natif en Java** avec les dernières APIs (ViewBinding, FusedLocationProviderClient, RecyclerView).
- **Firebase Realtime Database** pour la synchronisation GPS temps réel avec une latence inférieure à 500ms.
- **Firebase Authentication** pour la gestion sécurisée des comptes.
- **Google Maps SDK** et **Google Places API** pour les fonctionnalités cartographiques.
- Le développement backend avec **Google Cloud Platform**, **Firebase** et **Google Maps**.
- La conteneurisation avec **Android Studio**.
- La méthode agile **Scrum** appliquée à un projet de 6 développeurs.

Bilan méthodologique

La méthode Scrum a permis de structurer notre travail efficacement et de livrer des fonctionnalités de manière itérative. Le Product Backlog de 16 user stories a été entièrement complété en deux sprints de 3 semaines chacun. La collaboration via GitHub a fluidifié le travail en parallèle.

Bilan humain

Ce projet a été une formidable expérience de travail en équipe. La répartition claire des rôles, la communication régulière et l'entraide entre les membres de l'équipe ont été déterminantes dans la réussite du projet.

Perspectives

Des améliorations futures pourraient inclure :

— Amélioration de la précision des ETA avec des données de trafic en temps réel (Google Traffic API).
— Ajout d'un système de **notification push** (Firebase Cloud Messaging) pour alerter les passagers à l'approche d'un bus.
— Intégration d'un **système de paiement mobile** pour l'achat de tickets en ligne.
— Déploiement sur le **Google Play Store** pour une utilisation publique à Sousse.
— Intégration avec les données officielles de la **STS** (Société de Transport de Sousse).
— Développement d'une version iOS avec Flutter pour couvrir l'ensemble du marché mobile.

WinLkar représente une solution concrète et opérationnelle pour améliorer l'expérience des usagers des transports en commun à Sousse. Nous espérons qu'il constituera une base solide pour de futures extensions et pourra, à terme, contribuer à la modernisation des transports publics en Tunisie.

— *Fin du rapport* —

Bibliographie et Webographie

1. Android Developers. *Android Developer Documentation*. Google LLC, 2025. https://developer.android.com/docs
2. Firebase. *Firebase Realtime Database Documentation*. Google LLC, 2025. https://firebase.google.com/docs/database
3. Firebase. *Firebase Authentication Documentation*. Google LLC, 2025. https://firebase.google.com/docs/auth
4. Google. *Maps SDK for Android Developer Guide*. Google LLC, 2025. https://developers.google.com/maps/documentation/android-sdk
5. Google. *Places SDK for Android*. Google LLC, 2025. https://developers.google.com/maps/documentation/places/android-sdk
6. Schwaber K., Sutherland J. *The Scrum Guide*. Scrum.org, 2020. https://scrumguides.org
7. Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.
8. Booch, G., Rumbaugh, J., Jacobson, I. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.

Annexe A

Annexe A — Configuration Firebase

A.1 google-services.json

Le fichier `google-services.json` contient la configuration Firebase nécessaire à l'application Android. Il doit être placé dans le répertoire `app/` du projet. Ce fichier JSON est structuré autour de trois sections principales : `project_info` qui regroupe le numéro de projet, son identifiant unique (`winlkar-app`) et le nom du bucket de stockage Firebase. La section `client` contient les informations spécifiques à l'application Android, notamment le `mobilesdk_app_id` généré par Firebase et le `package_name` de l'application (`com.winlkarr.app`). Enfin, le champ `configuration_version` indique la version du schéma du fichier de configuration.

A.2 Règles de sécurité Firebase

Les règles de sécurité Firebase Realtime Database définissent les droits d'accès en lecture et en écriture pour chaque nœud de la base. Le nœud `activeTrips` est accessible en lecture publique, permettant à tous les passagers de consulter les positions des bus, mais l'écriture est restreinte aux utilisateurs authentifiés. Les nœuds `buses` et `drivers` requièrent une authentification pour la lecture, et l'écriture est réservée aux administrateurs identifiés par le rôle `admin` dans le token Firebase. Le nœud `feedbacks` autorise la lecture et l'écriture à tout utilisateur authentifié, qu'il soit passager, conducteur ou administrateur.

Annexe B

Annexe B — Description des Interfaces de l'Application

Cette annexe décrit les principales interfaces de l'application WinLkar telles qu'elles ont été implémentées lors du Sprint 1.

Écran d'accueil — Sélection du rôle : Au lancement de l'application, l'utilisateur est accueilli par un écran sobre sur fond sombre présentant trois cartes cliquables correspondant aux trois rôles disponibles : Passager (*Track buses in real-time*), Conducteur (*Manage and broadcast your trip*) et Administrateur (*System control panel*). Chaque carte affiche une icône colorée distinctive. Un badge vert **LIVE** dans le coin supérieur droit confirme la connexion active à Firebase.

Écran de liste des stations : Accessible depuis le rôle Passager, cet écran affiche la liste complète des arrêts de bus de la région de Sousse sous forme de cartes. Chaque élément indique le nom de la station et son statut (*No active buses* si aucun bus n'est en service). Une barre de recherche en haut de l'écran et un bouton *Open Map* permettent respectivement de filtrer les stations et d'afficher leur position sur la carte.

Écran de connexion du conducteur : Cet écran présente un formulaire de connexion composé d'un champ *Username* et d'un champ *Password* avec icône de visibilité. Un bouton *Sign In* soumet les identifiants à Firebase Authentication. En cas de succès, le conducteur est redirigé vers son tableau de bord de gestion de trajet.

Annexe C

Annexe C — Glossaire

API REST Interface de programmation applicative basée sur le style d'architecture REST (Representational State Transfer).

bcrypt Algorithme de hashage de mots de passe conçu pour être lent et résistant aux attaques par force brute.

ETA Estimated Time of Arrival — temps d'arrivée estimé.

Firebase Plateforme de développement mobile et web de Google.

GPS Global Positioning System — système de positionnement par satellite.

Haversine Formule mathématique permettant de calculer la distance entre deux points sur une sphère à partir de leurs coordonnées géographiques.

JWT JSON Web Token — standard ouvert d'authentification par jeton.

MVC Modèle-Vue-Contrôleur — patron d'architecture logicielle.

NoSQL Base de données non relationnelle.

Scrum Méthode agile de gestion de projet.

SDK Software Development Kit — kit de développement logiciel.